CENG493

Text Preprocessing and Vector Models

NLP Tasks Implementation Steps

1. Formulating a research question or NLP task.
2. Data collection and corpus building.
3. Data preprocessing.
4. Generating embeddings.
5. Embedding-related tasks.
6. Results.
7. Interpreting results.

NLP Task

- **Authorship attribution**: Can text analysis tools accurately attribute authorship to disputed or anonymous texts, and what features are most indicative of an author's style?
- **Topic modeling**: What topics are most relevant in a corpus? How does this vary by author or time-period?
- **Named entity recognition (NER)**: What locations, names, organizations, emotional expressions, and other "entities" are present in a corpus? When do characters co-occur in different chapters in a book?
- **Sentiment analysis**: How does sentiment evolve throughout a novel or a collection of poems, and what insights can be gained by analyzing the emotional tone of the text?
- **Literary style prediction**: How can I leverage predictive text AI such as ChatGPT to to analyze and predict changes in literary styles or genres over time?"
- **Information retrieval**: How can I retrieve relevant search results from a database?

https://carpentries-incubator.github.io/

Collect Data

- Evaluate data for corpus
  - Project needs
  - Rights and protections
  - Quality of data
  - Features of data
- Build corpus for analysis

Preprocessing

- Tokenization
- Lemmatization
- Remove stop words and punctuation
- Lowercase
- Parts of speech (POS) tagging

Text Embedding

Generate Numerical Representation of Text

- Bags of words (document)
- TF-IDF (document)
- Word2Vec, GloVe, FastText (word)
- Doc2Vec (document)
- BERT (word/document)

Text Embedding

Generate Numerical Representation of Text

- Bags of words (document)
- TF-IDF (document)
- Word2Vec, GloVe, FastText (word)
- Doc2Vec (document)
- BERT (word/document)

Embedding-Related Task

- **Authorship attribution**: Assess the stylistic similarities between different authors' works using embeddings (e.g., cosine similarity).
- **Topic modeling**: Utilize embeddings to uncover latent topics within a corpus and examine variations by author or time-period (e.g., Latent Semantic Analysis (LSA), Latent Dirichlet Allocation (LDA)).
- **Named entity recognition**: Extract and analyze named entities (locations, names, organizations, emotional expressions) within the corpus using embeddings (e.g., NER embedding).
- **Sentiment analysis**. Apply embeddings to track sentiment evolution throughout texts and derive insights from emotional tone analysis (e.g., sentiment analysis with embeddings).
- **Literary style prediction**: Utilize predictive text AI such as ChatGPT to predict changes in literary styles or genres over time, leveraging embeddings for analysis and prediction.
- **Information retrieval**: Employ embeddings for efficient retrieval of relevant search results from a database, enhancing search accuracy and relevance.

Results

- **Author attribution**: Assess the accuracy and effectiveness of authorship attribution techniques, identifying key features indicative of an author's style.
- **Topic modeling**: Determine the most relevant topics present in the corpus and explore variations by author or time-period.
- **Named entity recognition (NER)**: Identify and analyze entities present in the corpus, such as locations, names, organizations, and emotional expressions.
- **Sentiment analysis results**: Track sentiment evolution throughout texts and derive insights from emotional tone analysis.
- **Text prediction outcomes**: Analyze and predict changes in literary styles or genres over time using predictive text AI.
- **Information retrieval**: Retrieve relevant search results from a database, enhancing search accuracy and relevance.

Interpretation

- We consider how the results may inspire future directions of inquiry, such as conducting repeat analyses with different data cleaning methods, exploring related research questions, or refining the original research question based on the insights gained.
- This iterative process enables us to continually deepen our understanding and contribute to ongoing scholarly conversations.

So, how to represent text(s) in NLP?

Definitions

Sentence: a sequence of words. A sentence typically begins with a capitalized word and ends with punctuation (e.g. period, question mark)

Letters and characters

English is made up of 26 letters (A-Z)
A more general term: "character"
Characters can represent punctuation and whitespace too (e.g. "\n")
In C++, Java, and others, the "character" is a fundamental data type
Sometimes, our NLP models will be build on words, but other times, they will be in terms of characters

Definitions

Vocabulary: the set of "all" words
In most cases, vocabulary won't contain every possible word, but rather, just a "reasonable" subset of words
How many to keep? 1000s? 10k's? 100k's? 1M's?
It's your choice! Base it on experimental results

Corpus:

"A large collection of writings of a specific kind or on a specific subject."
"A collection of writings or recorded remarks used for linguistic analysis."
In this class, it will simply refer to our ML dataset

N-Gram

- Simply refers to N consecutive items (e.g. words, subwords, characters)
- Single (one) item: unigram
- Two items: bigram
- Three items: trigram
- Example use-case: the word2vec algorithm is just a fancy way to build a neural network for bigrams (really "skipgrams")
- Example use-case: Markov models consist of bigram probabilities

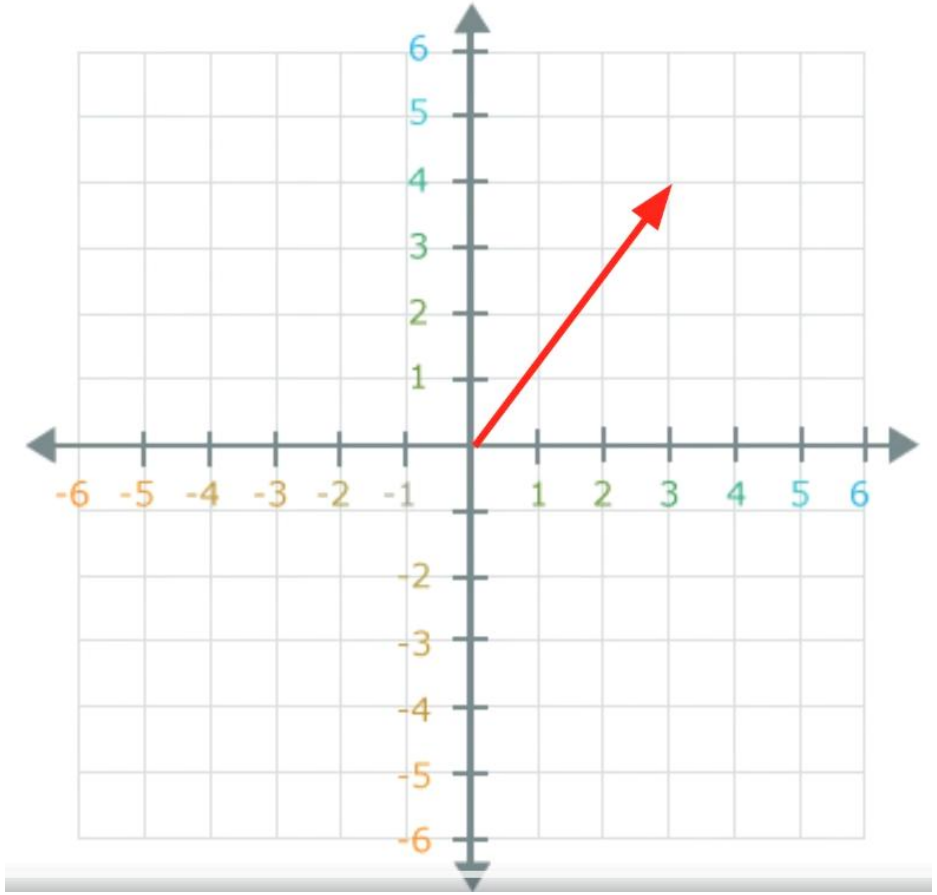
Text	N-gram
Data	1-gram
Great information	2-gram
I am fine	3-gram

What is a vector?

- A vector is a quantity that has both magnitude and direction
- Example: **velocity** (e.g. v = 10m/s NE)

Vector

- An array of scalars
- (3, 4) is shown on the right
- This vector has magnitude 5, and is 0.93 radians from the x-axis
- Cartesian vs. Polar coordinates
- Both require the same number of scalar components!
- Vectors can have 100s or 1000s of components. It is **easier** to think of each component as the "amount" on orthogonal axes



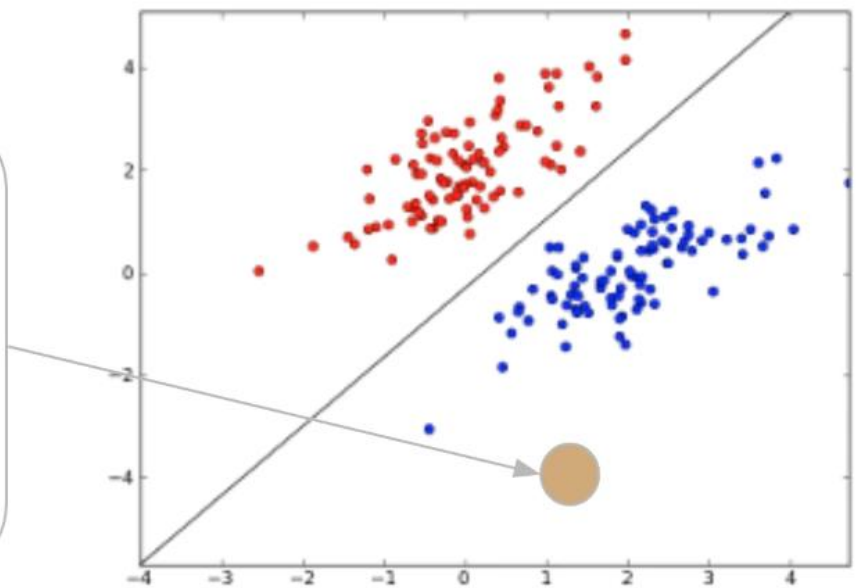
Why vector models?

Task: write a computer program that takes an email as input, and outputs whether or not the email is spam

Spam Detection, with Vectors

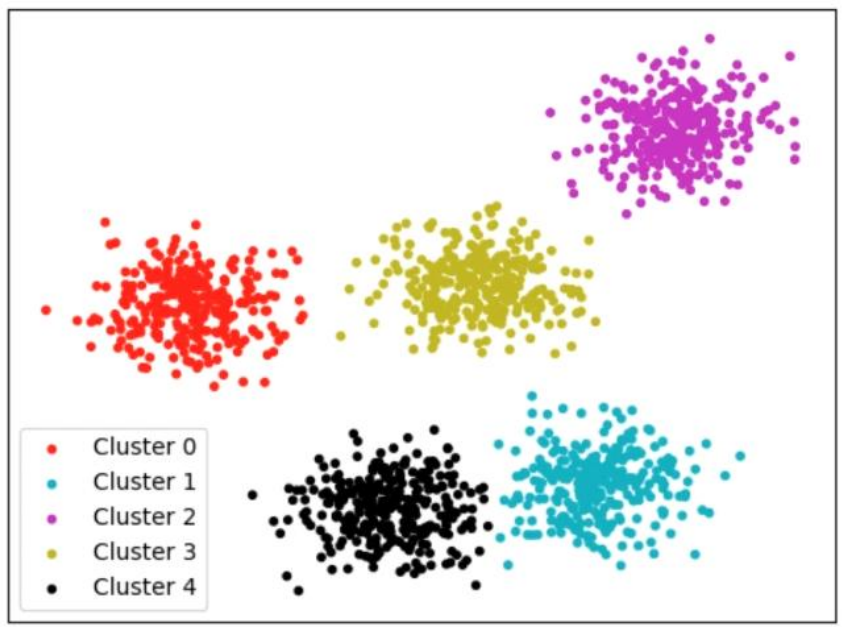
- This is just a preview, but suppose there was a way to **map** text into vectors, such that the vectors for each class fall into their own "clouds"?
- Then we can simply draw a **line** between the 2 clouds!

- Suppose we have a new email whose spam status is unknown
- The question is now simple: "Which side of the line is it on?"



Organizing Documents

- Suppose our business has a large collection of documents we need to organize, but there are way too many to read one-by-one
- We can automate this using **clustering**
- But only if we convert the documents to vectors first!



How?

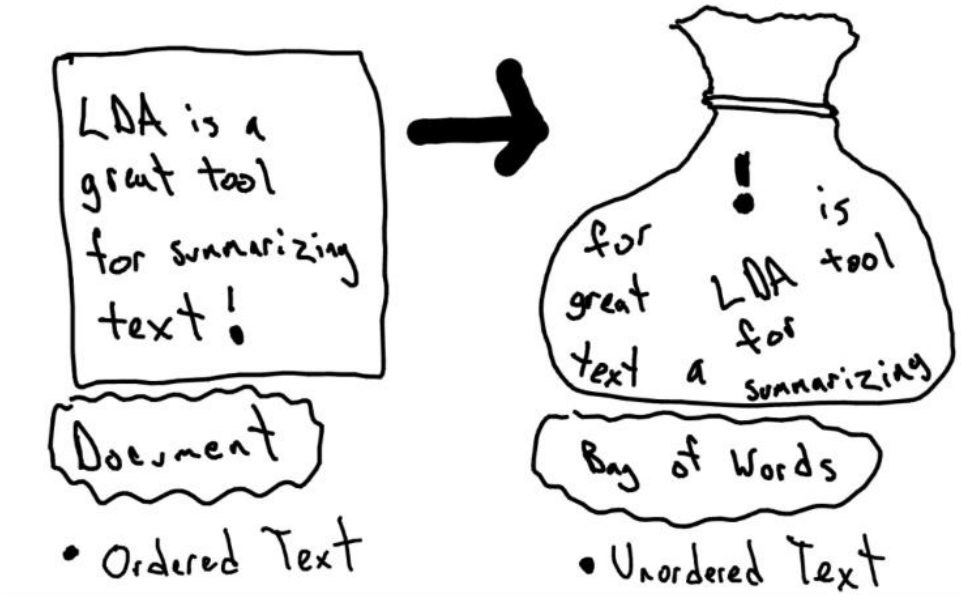
First very simple method → Bag of Words

- Text is **sequential**
- The specific **sequence** of words gives the text meaning
- If we **randomized** the words in a sentence, we would change its meaning, or more likely, make it incomprehensible
- Despite this, many NLP approaches do not consider word order
- We'll call these "bag of words" representations

But...

- Order matters! (all the time???)

- Vector models and classic machine learning models use bag of words
- Probabilistic models and deep learning do not
- This is just a rough categorization: we can still find probabilistic and neural models that also use bag of words



Documents -> Vectors

- Consider a simple classification task: discriminate between documents about biology and documents about physics
- But wait... what is a "document"?
- The dataset might consists of journal papers (1 paper = 1 document)
- Categorizing files on desktop (1 file = 1 document)
  - But some of those files could be journal papers too
  - They could also be whole books, simple text files / notes, etc.
- A document could just be a single sentence
- There are a wide range of possibilities!

- Let V = number of unique words in training corpus
- Each document will be converted into a vector of size V
- The vector components will contain the counts for each word

Documents -> Vectors

- Let V = number of unique words in training corpus
- Each document will be converted into a vector of size V
- The vector components will contain the counts for each word

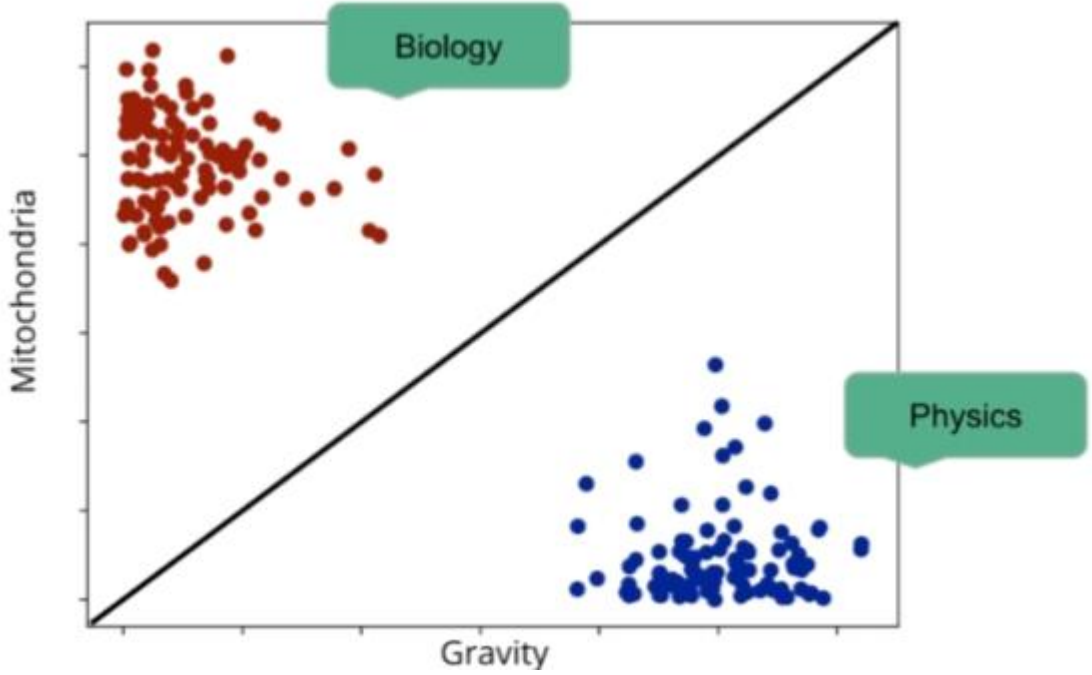
- V (vocabulary size) = 6

Each row is a vector of size 6

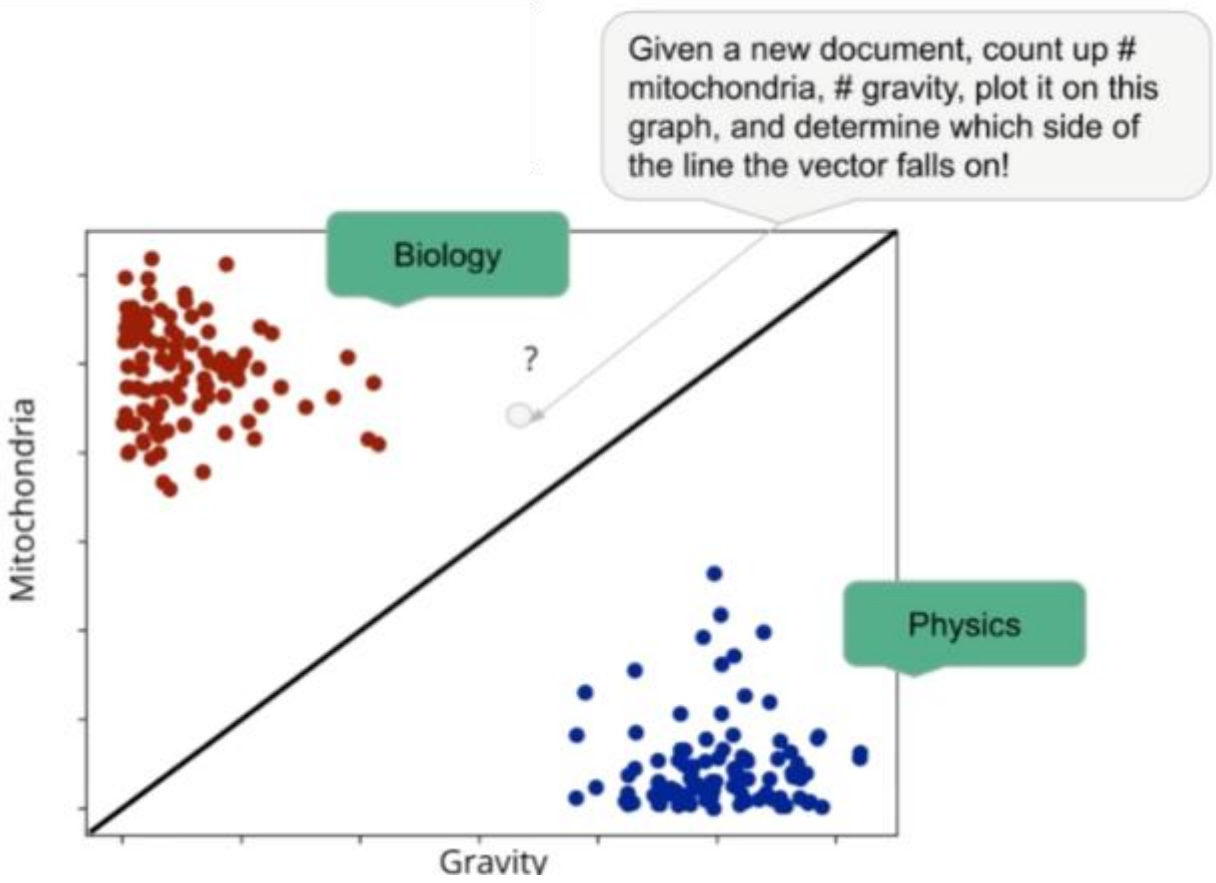
Original Documents
I like eggs
I hate cats
I like eggs and I like cats

and	cats	eggs	hate	I	like
0	0	1	0	1	1
0	1	0	1	1	0
1	1	1	0	2	2

Simple Classification Example



Simple Classification Example



Code

CountVectorizer in Code (SciKit-Learn)

```
vectorizer = CountVectorizer()

vectorizer.fit(list_of_documents_train)

Xtrain = vectorizer.transform(list_of_documents_train)

# all in one step

Xtrain = vectorizer.fit_transform(list_of_documents_train)

Xtest = vectorizer.transform(list_of_documents_test)
```

Normalization

- What if our corpus has very long and very short documents?
- Count vectors will have size disparities (more words = higher counts)

Method 1: Make the L2-norm 1

$$\hat{x} = \frac{x}{\|x\|_2}, \; where \; \|x\|_2 = \sqrt{\sum_{i=1}^{V} x_i^2}$$

Method 2: Divide by the sum (can think of each element as a probability)

Since counts are positive

$$\hat{x} = \frac{x}{\sum_{i=1}^{V} x_i}, \; note : \sum_{i=1}^{V} x_i = \sum_{i=1}^{V} |x_i| = \|x\|_1$$

Normalization

- A vector can be (length-) normalized by dividing each of its components by its length.

L_2 norm:

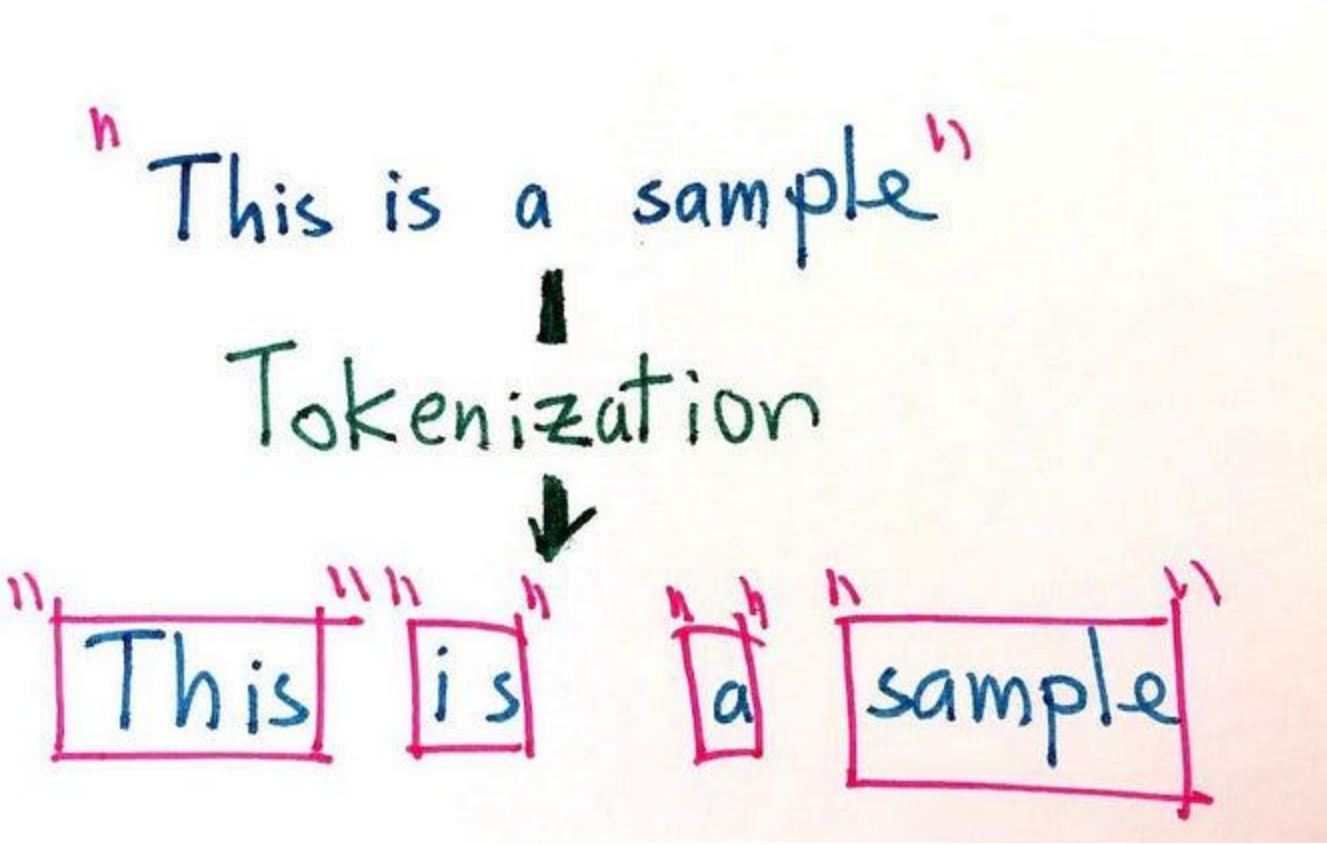
$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Dividing a vector by its L_2 norm makes it a unit (length) vector (on surface of unit hypersphere)
- Long and short documents now have comparable weights

But, first text preprocessing!

- **Text Cleaning**
  - **Converting to lowercase**
  - **Removing URLs**
  - **Removing emojis**
  - **Removing remove non-word and non-whitespace characters**
  - **Removing digits**
  - **Removing accents(?)**
- **Tokenization**
- **Stopword Removal**
- **Stemming**
- **Lemmatization**

Tokenization



Word-based tokenization

“Is it weird I don’t like coffee?” → [“Is”, “it”, “weird”, “I”, “don’t”, “like”, “coffee?”]

- **“?”** in **“coffee?”** is important or not?
  - What if we have a sentence **“I love coffee.”** in our corpus?
- The reason that we should take punctuation into account while performing tokenization is that we do not want our model to learn different representations of the same word with every possible punctuation.
- let’s take punctation into account.
  - **[“Is”, “it”, “wierd”, “I”, “don”, “’”, “t”, “like”, “coffee”, “?”]**

Word-based tokenization

- State-of-the-art NLP models have their own tokenizers because each model uses different rules to perform tokenization along with tokenizing using spaces.
- Tokenizers of different NLP models can create different tokens for the same text.
- Space and punctuation, and rule-based tokenization are all examples of word-based tokenization
- Drawbacks???

Character-based tokenization

- Character-based tokenizers split the raw text into individual characters.
- A language has many different words but has a fixed number of characters
- This results in a very small vocabulary.

Subword-based tokenization

- The main idea is to solve the issues faced by word-based tokenization;
  - very large vocabulary size
  - large number of OOV tokens
  - different meaning of very similar words
- and character-based tokenization;
  - very long sequences
  - less meaningful individual tokens
- The subword-based tokenization algorithms uses the following principles.
  1. Do not split the frequently used words into smaller subwords.
  2. Split the rare words into smaller meaningful subwords.

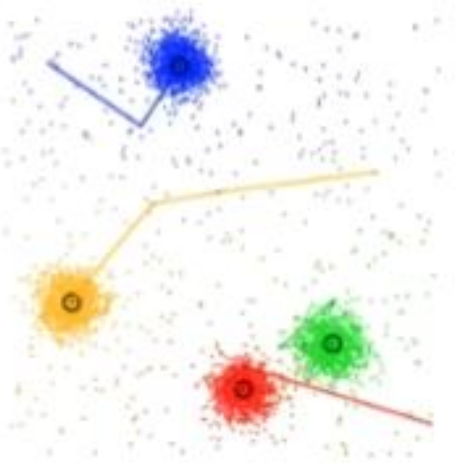
Stopwords

- Suppose we performed our counting procedure
- What happens with very common words? Such as: "and", "the", "it", "is", etc.
- Are these words useful?
- Any text will contain these words!

- High dimensionality is bad
- More dimensions → more computation
- Could it be beneficial to simply not include stopwords?

Stopwords – Distance consideration

- Recall: our understanding of feature vectors involves their distance to each other
- If we count all the "and"s, "the"s, etc. the vectors which will be *close* will simply be the ones with similar numbers of these words!
- They might overshadow more important words, like "mitochondria" or "voltage"
- Thus, **stopwords** are the words we wish to ignore



Stopwords

NLTK Stopwords

```
import nltk

nltk.download('stopwords')

from nltk.corpus import stopwords


stopwords.words('english')

stopwords.words('german')
```

Many languages are included, such as English, German, Spanish, and Arabic.

Check:
~/nltk_data/corpora/stopwords/

Stemming & Lemmatization

- With basic word tokenization, each variation of a word will have its own vector component: "walk", "walking", "walks", "walked", ...
- "Walk" is no closer to "walking" than it is to "cartwheel" (unless similarity can be learned from the data)
- This also leads to high-dimensional vectors
- Practical issue: imagine we're building a search engine (like DuckDuckGo)
- I search for "running" (What about "ran" or "run"? How to match them?)
- Solution: convert words to their root word

Stemming & Lemmatization

- Stemming is very **crude** - it just chops off the end of the word
- The result is not necessarily a real word

- Lemmatization is more sophisticated, uses actual rules of language
- The true root word will be returned

Stemming

- Based on simple heuristics
- Ex: Ends with SSES → Remove ES
- Ex: "Bosses" → "Boss"
- Ex: "Replacement" → "Replac" (not a real word)
- Multiple stemming algorithms (e.g. Porter Stemmer in NLTK)

```
from nltk.stem import PorterStemmer
porter = PorterStemmer()
porter.stem("walking") # returns 'walk'
```

Lemmatization

- Think of it as a lookup table / table of rules
- Stemming: "Better" → "Better"
- Lemmatization: "Better" → "Good"
- Note: "Was" is the past-tense of "Is", both are derivatives of "Be"
- Stemming: "Was" → "Wa"
- Lemmatization: "Was" / "Is" → "Be"
- Stemming: "Mice" → "Mice"

How to Use Lemmatization

- Appears in NLTK, spaCy, and others

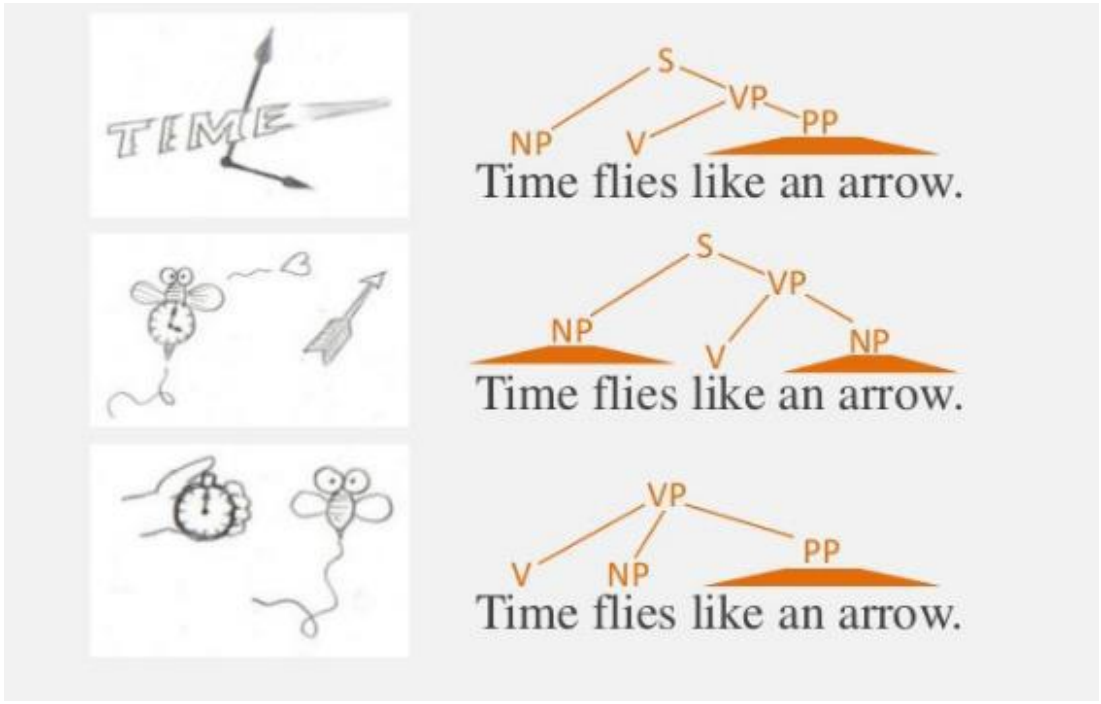
Note: "pos" stands for "parts-of-speech"

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
nltk.download("wordnet") # only need to do once

lemmatizer = WordNetLemmatizer()
lemmatizer.lemmatize("mice") # returns 'mouse'


lemmatizer.lemmatize("going") # returns 'going'
lemmatizer.lemmatize("going", pos=wordnet.VERB) # returns 'go'
```

Applications in NLP

- **Tokenization** is used in text preprocessing, sentiment analysis, and language modeling.
- **Stemming** is useful for search engines, information retrieval, and text mining.
- **Lemmatization** is essential for chatbots, text classification, and semantic analysis.

POS(Parts-Of-Speech) Tagging



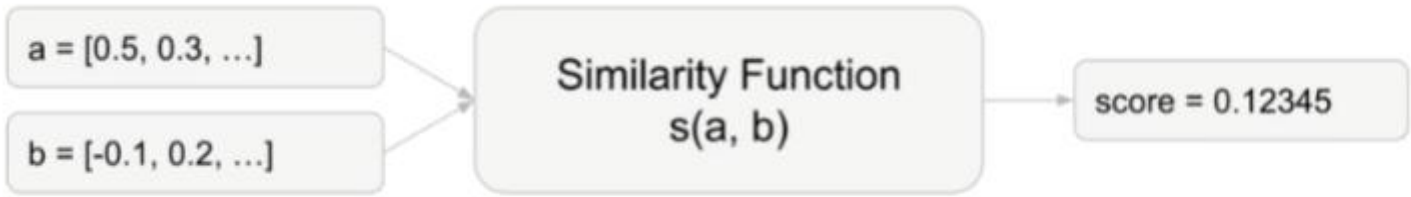
Why	not	tell	someone	?
adverb	adverb	verb	noun	punctuation mark, sentence closer

POS

- Part-of-speech tags describe the characteristic structure of lexical terms within a sentence or text
- We can use them for making assumptions about semantics.
  - Named Entity Recognition
  - Co-reference Resolution
  - Speech Recognition

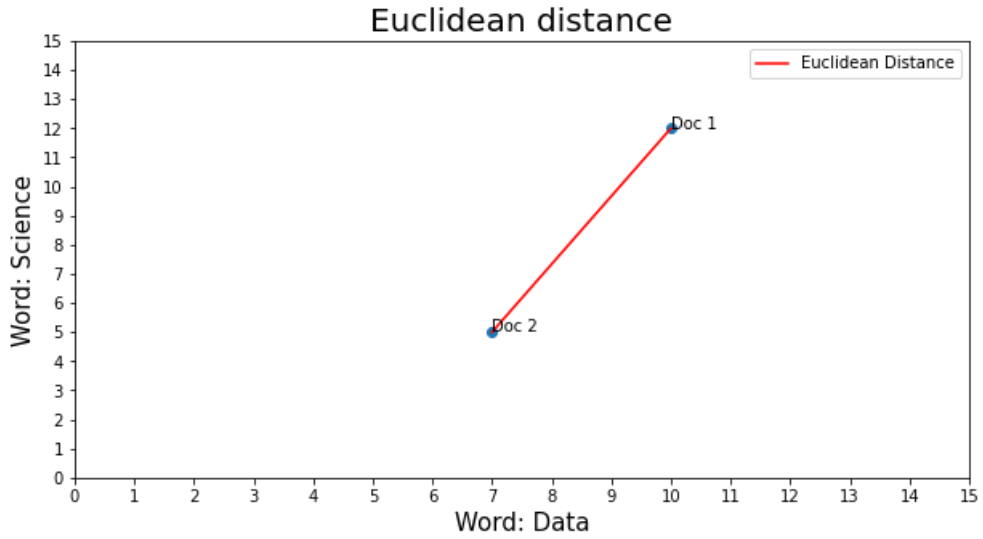
Back to vector models

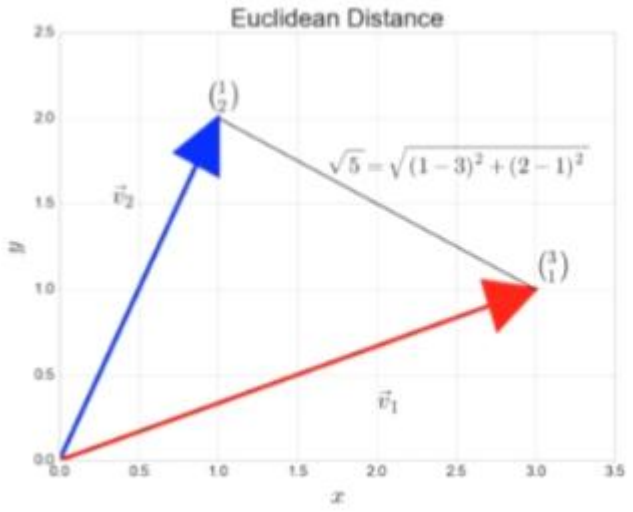
Vector Similarity



How is it useful? Given some document, find the "most similar" document

How can we calculate vector similarity?





Euclidean Distance

Text 1: I love ice cream

Text 2: I like ice cream

Text 3: I offer ice cream to the lady that I love

Text/Word	I	love	ice	cream	like	offer	to	the	lady
Text 1	1	1	1	1	0	0	0	0	0
Text 2	1	0	1	1	1	0	0	0	0
Text 3	1	1	1	1	0	1	1	1	1

$$dist(X,Y) = \sqrt{\sum_1^n (x_i - y_i)^2}$$

$$Dist(text: 1, text: 2) = \sqrt{(1-1)^2 + (1-0)^2 + (1-1)^2 + (1-1)^2 + (0-1)^2 + (0-0)^2 + (0-0)^2 + (0-0)^2 + (0-0)^2} \simeq 1{,}414$$

$$Dist(text: 1, text: 3) = \sqrt{(1-1)^2 + (1-1)^2 + (1-1)^2 + (1-1)^2 + (0-0)^2 + (0-1)^2 + (0-1)^2 + (0-1)^2 + (0-1)^2} \simeq 2$$

$$Dist(text: 2, text: 3) = \sqrt{(1-1)^2 + (0-1)^2 + (1-1)^2 + (1-1)^2 + (1-0)^2 + (0-1)^2 + (0-1)^2 + (0-1)^2 + (0-1)^2} \simeq 2{,}45$$

Any problem???

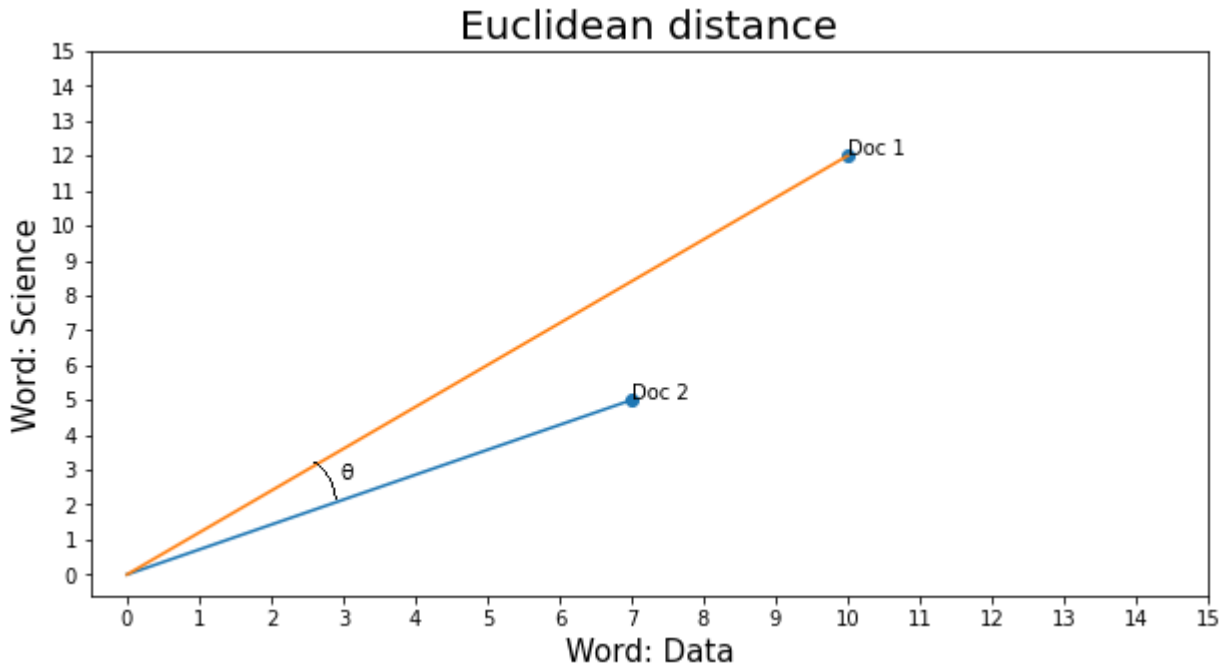
Another Example

- «I love playing football with my friends»
- «I hate waching and playing basketball»
- «When I was a kid I was playing football with my friends every day all the evening»

	all	and	basketball	day	evening	every	football	friends	hate	kid	love	my	playing	the	waching	was	when	with
text_1	0	0	0	0	0	0	1	1	0	0	1	1	1	0	0	0	0	1
text_2	0	1	1	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0
text_2	1	0	0	1	1	1	1	1	0	1	0	1	1	1	0	2	1	1

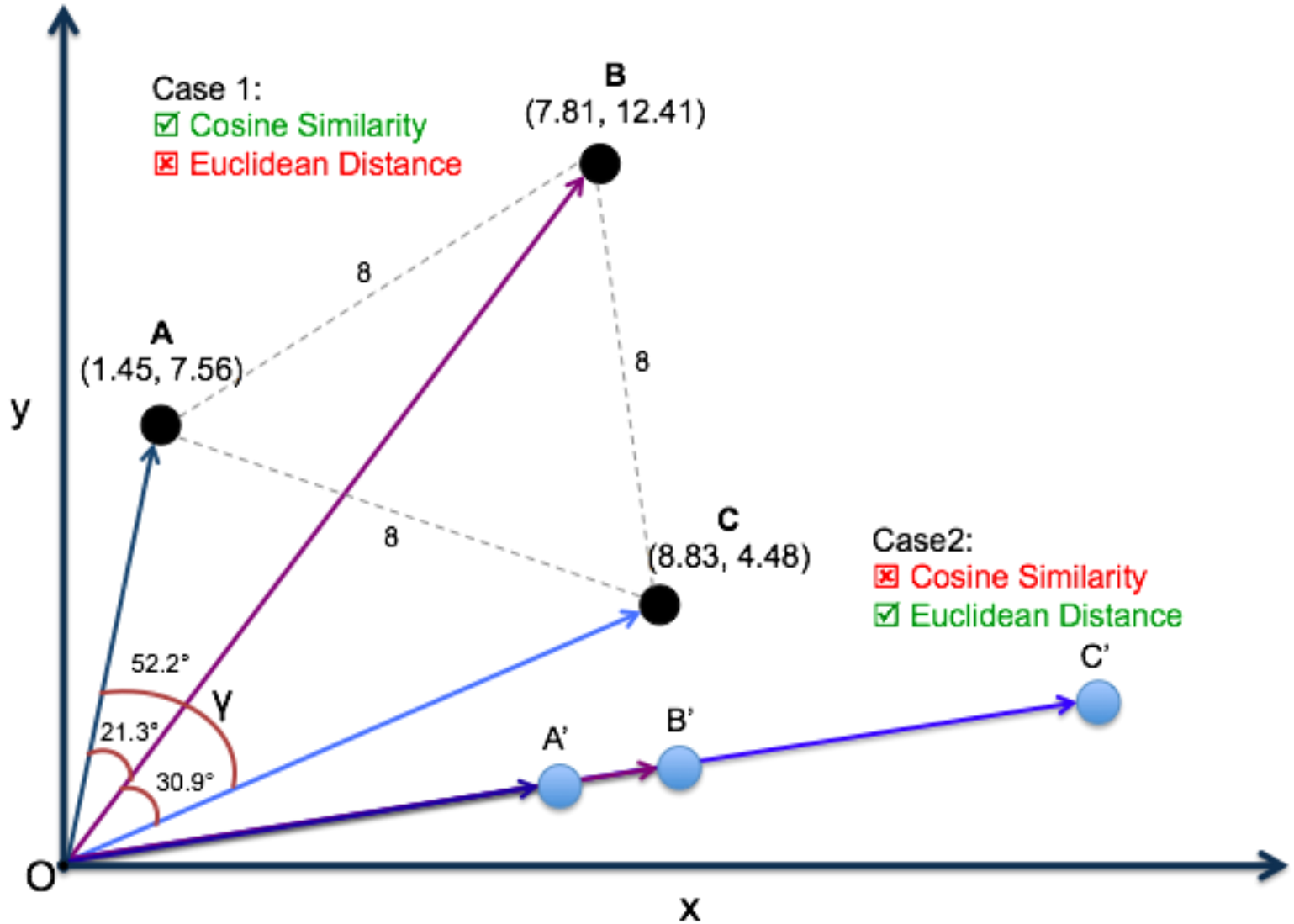
	Text_1	Text_2	Text_3
text_1	0.000000	3.000000	3.464102
text_2	3.000000	0.000000	4.358899
text_3	3.464102	4.358899	0.000000

Cosine similarity



$$sim(X,Y) = \cos(\theta_{x,y}) = \frac{X * Y}{||X|| * ||Y||} \equiv \frac{\sum_i^k X_i * Y_i}{\sqrt{\sum_i^k Xi^2}\sqrt{\sum_i^k Yi^2}}$$

$-1 \leq \cos(\theta_{x,y}) \leq 1$, But for documents: $0 \leq \cos(\theta_{x,y}) \leq 1$


Case 1:
☑ Cosine Similarity
☒ Euclidean Distance
B
(7.81, 12.41)
A
(1.45, 7.56)
C
(8.83, 4.48)
8
8
8
y
52.2°
γ
21.3°
30.9°
O
x
Case2:
☒ Cosine Similarity
☑ Euclidean Distance
C'
B'
A'

TF-IDF (Term Frequency-Inverse Document Frequency)

Still embedding?

TF-IDF (Term Frequency-Inverse Document Frequency)

- TF-IDF (Term Frequency-Inverse Document Frequency) is a method used to assess the importance of a word in a document or a collection of texts.
- The result of TF-IDF is obtained by multiplying Term Frequency (TF) and Inverse Document Frequency (IDF).

Term Frequency (TF):

$$\mathrm{TF}(t, \text{text}) = \frac{\text{Number of times term } t \text{ appears in text}}{\text{Total number of terms in the text}}$$

Inverse Document Frequency (IDF):

$$\mathrm{IDF}(t, \text{corpus}) = \log\left(\frac{\text{Total number of texts in corpus } |\text{corpus}|}{\text{Number of texts containing term } t}\right)$$

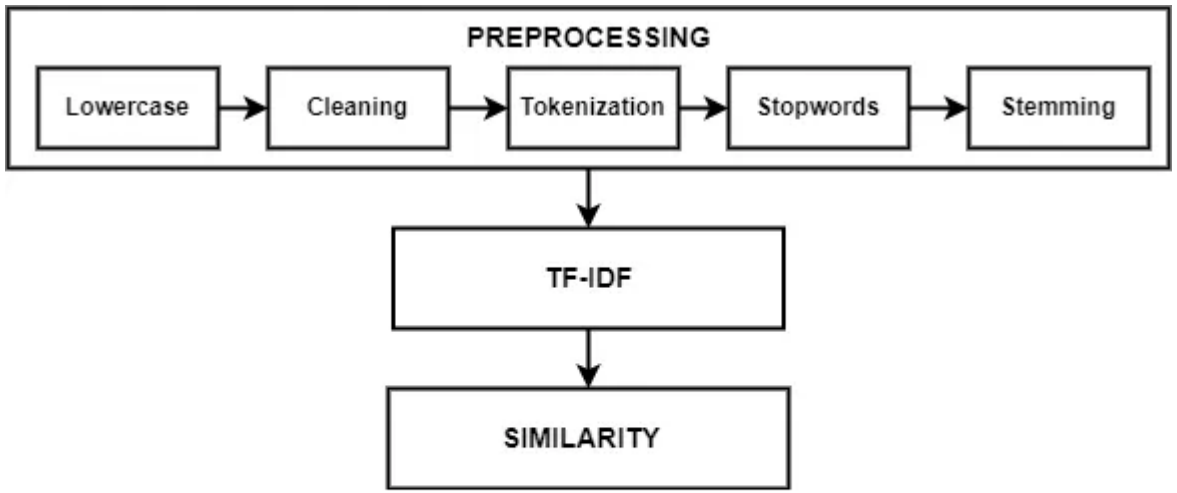
TF-IDF:

$$\text{TF-IDF}(t, \text{text}, \text{corpus}) = \mathrm{TF}(t, \text{text}) \times \mathrm{IDF}(t, \text{corpus})$$

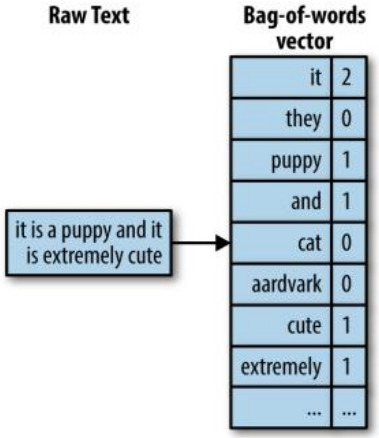
TF-IDF

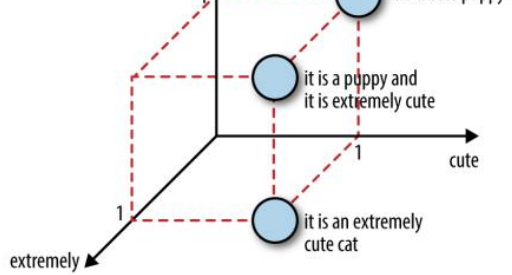
- Term Frequency (TF):
  - Measures how often a term appears in a text.
  - It is calculated as the number of occurrences of a term divided by the total number of terms in the text.
- Inverse Document Frequency (IDF):
  - Measures the rarity of a term across all texts in the collection.
  - It is calculated as the logarithm of the total number of texts divided by the number of texts containing the term.
- TF-IDF:
  - The product of TF and IDF, giving a higher weight to terms that are frequent in a text but rare in the entire .

TF-IDF


PREPROCESSING
Lowercase
Cleaning
Tokenization
Stopwords
Stemming
TF-IDF
SIMILARITY

BoW vs TF-IDF





We can reduce the feature space by applying stopwords removal, stemming etc. but the initial step of ***bag-of-words*** acts as a downside because it emphasizes words only based on counts.

BoW vs TF-IDF

- Unlike, bag-of-words, **tf-idf** creates a normalized count where each word count is divided by the number of documents this word appears in.
  - **bow(w, d) = # times word w appears in document d.**
  - **tf-idf(w, d) = bow(w, d) x N / (# documents in which word w appears)**
- The fraction (N / (# documents in which word w appears)) is known as **inverse document frequency.**